PROJECT DRYDOCK

The Chronicle Verification and Simulation System

Governing Architecture and Product Requirements Document

Governing Principle

A Chronicle may not be published merely because its files are syntactically valid. It must be typed, structurally coherent, semantically reachable, safely simulatable, accessible, version-compatible, and proven against declared scenarios using the same canonical runtime rules that govern real play.

Version 1.0 • July 26, 2026

Prepared for the Chronicles platform

Document Control

Field	Value
Program	Project Drydock
Subsystem	The Chronicle Verification and Simulation System
Document type	Governing architecture and product requirements
Version	1.0
Date	July 26, 2026
Status	Governing baseline
Repository baseline reviewed	Kgray44/treasurehuntSoT at 676b21ed030a5470d4ea0a36c0688ed3ecb161e5
Primary product surfaces	Creator Studio, Chronicle publishing, preview sandbox, validation reports, CLI and CI
Core implementation rule	One typed authoring model, one validation authority, and one simulation adapter over canonical runtime semantics

Contents

1. Executive Summary
2. Project Identity and Product Vision
3. Current Repository Context
4. Non-Negotiable Design Principles
5. Scope and Non-Goals
6. Canonical Architecture and Ownership
7. Authoring Contract and Schema Registry
8. Typed Block Schemas
9. Variable Registry and State Model
10. Expression and Condition System
11. Graph and Control-Flow Model
12. Static Analysis and Content Linting
13. Media, Accessibility, Privacy, and Provider Validation
14. Deterministic Simulation Architecture
15. Scenario Model and Coverage
16. Fault Injection and Environmental Simulation
17. Runtime Fidelity and Differential Proof
18. Creator Studio Experience
19. Validation Reports, Severity, and Repairs
20. Publishing Gates, Evidence, and Waivers
21. Schema Evolution and Historical Compatibility
22. Cross-Project Integration Boundaries
23. Data Model and Service Contracts
24. Security and Privacy
25. Accessibility
26. Performance and Operational Architecture
27. Testing and Acceptance Matrix
28. Implementation Phases
29. Final Acceptance Criteria
30. Recommended Technical Baseline
31. Governance and Change Control
32. Glossary
33. References

1. Executive Summary

Project Drydock establishes the Chronicles platform as a rigorous authoring-verification, content-linting, and deterministic simulation system. It ensures that Creator Studio can prove a Chronicle is structurally valid, type-safe, reachable, accessible, compatible, and behaviorally coherent before that Chronicle becomes an immutable published version or a live Voyage.

The current platform already contains a strong foundation: 23 governed Story Block types, immutable publishing, graph connections, asset checks, reachability validation, completion-path analysis, preview sessions, and a canonical One Voyage progression engine. Drydock does not replace that foundation. It turns those pieces into one comprehensive engineering and Creator-facing system instead of leaving correctness distributed among field metadata, special-case validators, browser previews, and the Creator's remaining supply of optimism.

The system combines strict versioned block schemas, a typed variable and expression model, whole-graph analysis, actionable lint rules, deterministic scenario execution, virtual time, provider simulation, fault injection, coverage evidence, publishing gates, and historical compatibility. It must support ordinary Creators through clear explanations and safe repair guidance while remaining precise enough to serve as a release gate in automated validation and Community Exchange workflows.

> **Core Product Outcome**
>
> A Creator can build a Chronicle, see exactly what is wrong and why, exercise every meaningful branch and failure state, compare expected and actual outcomes, and publish only when the Chronicle has a complete, reproducible verification receipt. The platform stops treating "I clicked through it once" as a formal test strategy.

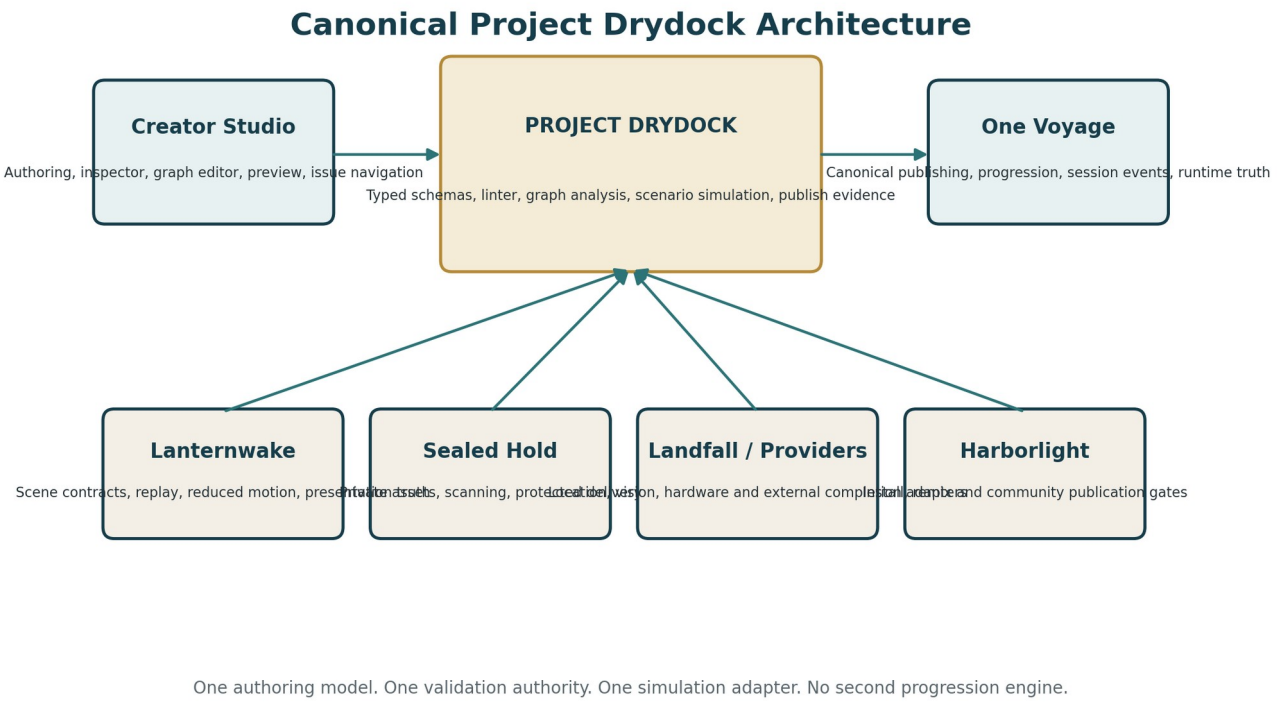

Figure 1. Project Drydock sits between Creator Studio and canonical One Voyage behavior while consuming narrow contracts from adjacent projects.

2. Project Identity and Product Vision

2.1 Program Name

The subsystem is named Project Drydock.

A drydock is where a vessel leaves ordinary operation so its structure can be inspected, stressed, repaired, and cleared before launch. The name fits a system that examines a Chronicle beneath its visual surface: schemas, references, branches, state transitions, assets, fallbacks, runtime behavior, and compatibility. It also avoids naming the project “Validation Final Final,” a phrase that has never survived contact with version control.

2.2 Formal Subtitle

The Chronicle Verification and Simulation System.

2.3 Product Vision

Drydock should make sophisticated Chronicle authoring safer without making it feel like a compiler course was hidden inside a date-night application. Creators should receive understandable guidance, visual issue locations, reproducible scenario results, and clear publication readiness. Advanced users should be able to inspect exact rule codes, state traces, coverage, schema versions, and runtime differences.

The system should shift the Creator experience from reactive debugging to guided proof. The platform should detect invalid values before save where possible, explain whole-Chronicle problems before publication, and reproduce failures in a safe sandbox rather than requiring a live Player, Captain, database, and favorable alignment of several celestial bodies.

2.4 Primary Use Cases

- Validate a simple linear Chronicle before publishing.
- Prove every branch in a complex choice-driven Chronicle reaches a valid ending.
- Detect impossible conditions, undeclared variables, wrong value types, and unsafe state operations.
- Simulate Captain approvals, timers, text-answer outcomes, verification providers, artifacts, side quests, and finale conditions.
- Inject offline, reconnect, asset-failure, provider-unavailable, stale-event, duplicate-event, and cancellation behavior.
- Prove private or Captain-only content never enters Player-safe projections.
- Validate Community-installed or remixed content before it enters a Creator draft or becomes a new release.
- Preserve old published versions by upcasting their schemas and proving their runtime compatibility.
- Generate machine-readable evidence for CI, implementation agents, support operators, and publishing workflows.

2.5 Product Non-Goals

- Drydock is not a second Chronicle progression engine.
- Drydock is not a general-purpose programming language or arbitrary script host.
- Drydock does not guarantee that a story is emotionally compelling, funny, romantic, or immune to a Creator writing seventeen paragraphs where one clue would suffice.
- Drydock does not replace real-device testing for geolocation, camera, 3D, audio, accessibility, or performance providers.
- Drydock does not mutate live Tale Sessions or fabricate production events.
- Drydock does not replace Lanternwake presentation validation, Sealed Hold asset security, Landfall field tests, or Relic Forge model validation; it coordinates their declared contracts.
- Drydock does not automatically repair destructive structural problems without explicit Creator approval and an undo path.
- Drydock does not permit arbitrary JavaScript, remote code, or unbounded expressions in Chronicle content.

3. Current Repository Context

This governing baseline was prepared against repository commit 676b21ed030a5470d4ea0a36c0688ed3ecb161e5, which records the final Project Lanternwake mainline closure. Current implementation must always revalidate the fetched remote repository before work begins; the SHA documents the architecture reviewed, not a future command to restore the past.

The current Story Block registry declares 23 block types and a `schemaVersion` of 1. Its common schema builder accepts a broad record of unknown values and primarily verifies required-field presence. This is a reasonable Phase 1 authoring foundation, but it does not by itself enforce every field type, enum, range, nested shape, cross-field invariant, or provider-specific rule.

The current draft validator already provides important whole-Chronicle protections: unknown block detection, required answers, asset existence and media-role checks, future-provider rejection, broken connection detection, choice and condition target checks, unreachable block detection, completion-path analysis, and unused-asset warnings. Drydock must preserve these successful behaviors, move them into a coherent versioned rule architecture, and expand them rather than replacing the validator with a decorative green checkmark.

Current capability	Existing strength	Drydock extension
Block registry	Stable type IDs, defaults, inspector fields, schema version, asset declarations	Strict configuration, presentation, completion, and cross-field schemas per block version
Draft validation	Assets, answers, references, reachability, completion paths, future-provider checks	Incremental rule graph, variable analysis, cycle/state analysis, actionable repairs, evidence
Preview sessions	Safe draft preview and play-from-here behavior	Deterministic scenarios, virtual time, faults, assertions, coverage and replayable traces
Publishing	Immutable checksummed snapshots and graph validation	Required verification receipt, schema compatibility matrix, waivers and CI gate
One Voyage	Canonical progression and event truth	Simulation adapter and differential proof without duplicate business logic
Lanternwake	Presentation truth, replay, reduced motion, lifecycle ownership	Scene-reference linting and scenario assertions through declared contracts

Repository Rule

Drydock must be implemented as an extension of the accepted canonical Chronicle architecture. It must not create a third authoring model, a simulator-only block format, or a friendlier duplicate of One Voyage semantics.

4. Non-Negotiable Design Principles

Principle	Requirement
One Authoring Contract	Creator Studio, validators, preview, simulator, publishing, imports, and historical readers consume the same versioned block contracts.
Runtime Fidelity	Simulation delegates progression decisions to canonical One Voyage services or an exact pure adapter. A simulator that “mostly agrees” is a bug generator with excellent posture.
Strict but Understandable	Machine precision and Creator clarity are both required. Every issue has a stable code, exact location, explanation, and safe remediation guidance.
Determinism	The same Chronicle version, scenario, seed, virtual clock, provider outcomes, and engine version produce the same trace and result.
Incremental Feedback	Local edits receive fast local validation; full static and simulation gates remain explicit. Creators should not wait through an ocean voyage to learn a number was negative.
No Arbitrary Code	Conditions and state changes use typed ASTs and governed operations. No uploaded scripts, `eval`, dynamic imports, or remote execution.
Evidence, Not Theater	A green UI state exists only after authoritative validation or simulation evidence exists. Animation success never substitutes for content correctness.
Version-Pinned History	Published versions retain their schema versions and verification evidence. New rules do not silently reinterpret old Chronicles.
Fail Closed on Privacy	Unpublished prose, accepted answers, Captain notes, private variables, and protected media never enter unauthorized reports, logs, previews, or Community projections.
Bounded Exploration	Graph and state exploration use explicit limits, diagnostics, and safe incomplete results rather than hanging until the heat death of the workstation.

Principle	Requirement
Accessible Authoring	Issue navigation, graph state, coverage, simulation controls, and repairs work without color, drag-and-drop, sound, or motion as the only signal.
Composable Providers	Landfall, Vision Waypoint, hardware, artifacts, and future providers expose typed validation and simulation contracts rather than injecting custom logic into Drydock.

> **Hard Boundary**
>
> Drydock may validate and simulate a completion provider, but only One Voyage may create the authoritative progression event. Provider evidence does not receive a secret side door labeled “advance anyway.”

5. Scope and Non-Goals

5.1 Governed Scope

- Typed schemas for block configuration, presentation, completion, connections, provider options, and reusable component inputs.
- Variable declarations, scopes, operations, references, type analysis, rename propagation, and usage indexing.
- A typed condition and expression AST with deterministic evaluation and static validation.
- Canonical graph representation and control-flow analysis across chapters, choices, conditions, optional branches, loops, and terminal states.
- Static lint rules for structure, state, assets, accessibility, privacy, providers, performance, compatibility, and publish readiness.
- Deterministic simulation with virtual time, seeded outcomes, canonical progression semantics, state traces, and assertions.
- Creator-authored and generated scenarios, scenario suites, coverage maps, expected outcomes, and regression baselines.
- Fault injection for network, assets, providers, reconnect, duplicate delivery, cancellation, viewport, motion, and sound behavior.
- Creator Studio integration: issue navigator, variable explorer, graph overlays, scenario laboratory, state inspector, trace timeline, coverage, and publish gate.
- CLI and CI commands for validation, simulation, compatibility, fixture replay, and machine-readable reports.
- Versioned schema migration registry and frozen historical compatibility corpus.
- Immutable publishing evidence and governed waivers.

5.2 Explicit Exclusions

- Writing or rewriting the Creator’s story.
- Generating secret accepted answers or hidden clue content.
- Running a public AI service against private Chronicle prose without a separately governed privacy boundary.
- Authoritative Tale Session mutations.
- Live player analytics or behavior profiling beyond test evidence.
- Physical-world accuracy claims not supported by Landfall or a provider-specific field test.
- 3D parser implementation, malware scanning, object storage, or media transcoding owned by Relic Forge, Harborlight, or Sealed Hold.
- New animation runtimes or local GSAP scenes owned outside Lanternwake.
- Unbounded model checking across arbitrary state spaces.
- Automatic waiver of blocker issues.
- Public display of validation details that reveal private story structure or answers.

6. Canonical Architecture and Ownership

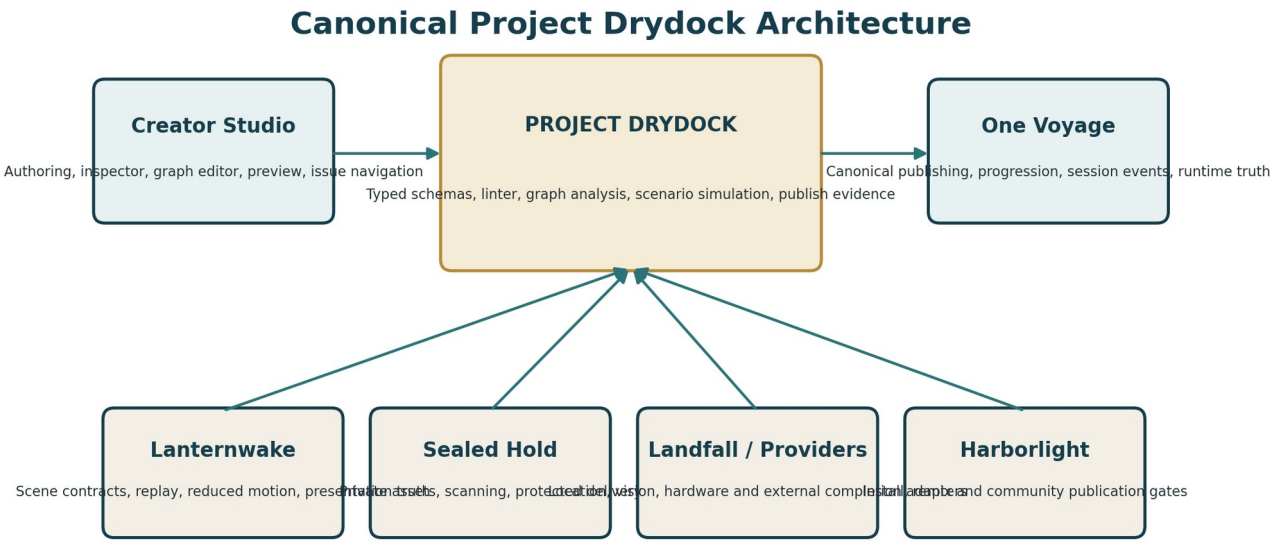

Figure 2. Drydock is a validation and simulation authority over authored content, not a replacement owner for publishing or progression.

6.1 Aggregate Boundaries

Aggregate	Owns	Must not own
Drydock Schema Registry	Versioned authoring schemas, canonical parsing, defaults, migrations, field metadata	Live session state or Creator identity
Drydock Validation Run	Rule version, issue set, source watermark, report digest, timing and status	Published content mutation
Drydock Scenario	Initial facts, scripted outcomes, virtual time, assertions, environment policy	Real credentials, real provider secrets, live user behavior
Drydock Simulation Run	Deterministic trace, final state, assertions, coverage, runtime adapter version	Authoritative TaleSession or TaleSessionEvent records
Drydock Publishing Evidence	Validated source checksum, schema versions, required suite results, waivers	Published snapshot contents or publication authority
One Voyage	Progression rules, runtime state, commands, ordered events, idempotency	Drydock issue ownership or Creator repairs
Creator Studio	Draft editing, UI state, human choices, save/autosave	Independent validation semantics

6.2 Required Service Boundaries

- **DrydockSchemaService -** registers, parses, canonicalizes, migrates, and describes versioned authoring contracts.
- **DrydockValidationService -** runs incremental and full rule sets against an immutable draft snapshot.
- **DrydockGraphAnalyzer -** builds control flow, reachability, termination, loop, dependency, and coverage models.
- **DrydockVariableService -** indexes declarations and references and performs type, scope, and operation analysis.
- **DrydockExpressionService -** validates and evaluates the governed condition AST.
- **DrydockSimulationService -** executes deterministic scenarios through the canonical runtime adapter.
- **DrydockScenarioService -** creates, versions, imports, runs, compares, and organizes scenario suites.
- **DrydockFaultService -** provides bounded, typed environmental faults and provider outcomes.

- **DrydockEvidenceService -** stores sanitized reports, coverage, traces, digests, and immutable publish evidence.
- **DrydockPublishingPolicy -** determines whether required rules, scenarios, compatibility checks, and waivers permit publication.
- **DrydockStudioProjection -** provides Creator-safe issue, graph, trace, and coverage views without leaking protected content.

6.3 Runtime Ownership

One Voyage owns progression transitions, event semantics, idempotency, inventory, variables, reveal state, membership, and completion. Drydock may invoke a pure or sandboxed adapter to those rules. It may never “simplify” them into a separate simulator engine because maintaining two definitions of truth is how software acquires folklore.

7. Authoring Contract and Schema Registry

7.1 Contract Shape

Every Story Block version must declare separate schemas for configuration, presentation, completion, connection requirements, asset references, provider requirements, accessibility obligations, and simulation behavior. A single broad JSON record may remain the database transport representation, but only after strict parsing into a versioned typed contract.

```
type DrydockBlockContract = {
  type: string;
  schemaVersion: number;
  configurationSchema: ZodType;
  presentationSchema: ZodType;
  completionSchema: ZodType;
  connectionPolicy: ConnectionPolicy;
  assetRequirements: AssetRequirement[];
  variableReads: VariableReferenceRule[];
  variableWrites: VariableWriteRule[];
  providerContract?: ProviderContractRef;
  accessibilityRules: AccessibilityRuleRef[];
  simulationAdapter: BlockSimulationAdapter;
  migrations: BlockSchemaMigration[];
};
```

7.2 Registry Requirements

- Stable block type IDs and monotonically increasing schema versions.
- No duplicate type/version pair.
- Explicit current version and minimum supported reader version.
- Strict unknown-field policy with only governed extension namespaces permitted.
- Defaults applied when a block is created or migrated, not silently during publishing.
- Canonical serialization with deterministic property ordering for checksums and diffs.
- Safe inspector metadata generated from or cross-validated against schemas.
- Machine-readable field paths and human-readable Creator labels.
- Cross-field validators for conditional requirements and incompatible combinations.
- Compatibility declaration for runtime, simulator, Studio, Community packages, and historical readers.

7.3 Canonicalization

Parsing produces a canonical value. Equivalent inputs such as trimmed labels or normalized enum aliases must become one representation before checksums, comparisons, or scenarios. Canonicalization must never rewrite Creator prose, accepted-answer capitalization policy, or private meaning without an explicit migration.

7.4 Extension Policy

Third-party or future project extensions may declare namespaced fields only through a registered contract. Unrecognized namespaced fields fail validation or remain preserved-but-inactive according to explicit compatibility policy. A field called `futureOptions` is not an architecture; it is a drawer where type safety goes to hide.

8. Typed Block Schemas

8.1 Required Validation Classes

Class	Examples
Primitive type	String, boolean, integer, finite number, identifier, timestamp, duration
Enum	Display mode, completion mode, transition, provider type, variable operation
Range	Volume 0-1, focal point 0-100, duration limits, nonnegative counts
Array shape	Choices, accepted outcomes, hints, component IDs, provider fallbacks
Nested object	Alignment, animation settings, route policy, artifact representation, completion configuration
Identity/reference	Asset ID, location ID, artifact ID, block ID, variable ID, scene name
Cross-field	Autoplay requires accessible fallback; provider mode requires provider options; model requires poster fallback
Contextual	Captain-only asset cannot appear in Player content; optional chapter rules differ from required paths
Operational	Configured provider available for publication; feature flag and runtime capability compatible

8.2 Configuration, Presentation, and Completion

These concerns must remain distinct even when stored together. Configuration defines content and behavior. Presentation defines appearance and allowed scene references. Completion defines how the canonical runtime may progress. This separation allows Drydock to prove that changing a paper style does not alter completion semantics and that a visually dramatic scene cannot silently become an answer validator.

8.3 Block-Specific Requirements

- **Narrative and notes:** nonempty governed text, safe formatting, readable completion policy, media alternatives.
- **Riddles and text answers:** nonempty accepted-answer set, normalization policy, no answer leakage, bounded hints, safe retry behavior.
- **Choice:** stable option IDs, unique labels where required, canonical graph edges, all targets valid, no duplicate representation in configuration and connections.
- **Condition:** typed expression AST, valid operands, explicit success/failure edges, deterministic evaluation.
- **Set Variable:** declared variable, compatible operation and value, scope authority, no arbitrary object mutation.
- **Media:** ready asset variant, MIME/role compatibility, captions/transcripts/alt text, size and performance classification, poster/fallback.
- **Artifact Reveal:** valid artifact definition, recipient policy, representation fallback, idempotent grant expectations, assembly compatibility where relevant.
- **Wait and timer:** bounded duration, virtual-clock behavior, cancellation and reconnect policy.
- **Provider completion:** registered provider, typed options, fallback, evidence retention, Captain override policy, simulation adapter.
- **Chapter and Voyage completion:** valid terminal semantics, no downstream automatic edge, outcome identity, repeat protection.

8.4 Unknown and Historical Blocks

Unknown block types remain visible as diagnostics and may not execute. Historical known types must be upcast through the compatibility registry or opened in a governed read-only mode. Drydock must distinguish “unsupported by this application version” from “corrupt,” because those are different problems and deserve different human panic levels.

9. Variable Registry and State Model

9.1 Purpose

Creators must declare state rather than typing arbitrary variable names into disconnected inspectors. The registry provides stable IDs, readable names, types, scopes, defaults, descriptions, allowed operations, privacy class, and usage references. The runtime remains owned by One Voyage; Drydock owns authoring verification and simulation of declared state.

9.2 Supported Core Types

Type	Permitted operations	Examples
Boolean	assign, toggle	doorOpened, finaleReady
Integer	assign, increment, decrement, min, max	hintCount, piecesFound
Number	assign, increment, decrement, min, max	score, routeProgress
String	assign, clear, compare	selectedName, clueMode
Enum	assign from declared members, compare	weatherState, chosenRoute
String set	add, remove, contains, count	visitedLocations, revealedClues
Identifier reference	assign or clear governed entity ID	selectedArtifact, activeWaypoint

Object and arbitrary JSON variables are excluded from the core model. A project needing structured state must declare a versioned custom type with strict schema and operations. Otherwise every variable eventually becomes `data: {}` and the simulator becomes a detective.

9.3 Scopes

- Chronicle definition scope for constants and reusable authored values.
- Session scope for the current shared Voyage state supported by One Voyage.
- Chapter scope for temporary governed state when runtime support exists.
- Player or crew-member scope only when a canonical Crewline or One Voyage contract exists; Drydock must not create member state by itself.
- Captain-only and Creator-only values are classifications and projection rules, not hidden client fields.

9.4 Usage Analysis

- Undeclared references and duplicate declarations.
- Type-incompatible comparisons and assignments.
- Reads before any possible initialization.
- Writes that can never be reached.
- Variables written but never read; variables read but never written.
- Constant conditions and impossible enum branches.
- Rename propagation across expressions, blocks, scenarios, assertions, and documentation labels.
- Privacy projection violations and Creator/Captain-only state entering Player presentation.

10. Expression and Condition System

10.1 Governed AST

Conditions use a versioned abstract syntax tree, not raw JavaScript or freeform text. The AST supports only declared operators, typed operands, deterministic evaluation, bounded nesting, and canonical serialization.

```
type Expression =
  | { kind: "literal"; valueType: ValueType; value: unknown }
  | { kind: "variable"; variableId: string }
  | { kind: "compare"; operator: CompareOperator; left: Expression; right: Expression }
  | { kind: "logical"; operator: "and" | "or"; operands: Expression[] }
  | { kind: "not"; operand: Expression }
  | { kind: "contains"; set: Expression; value: Expression }
  | { kind: "count"; source: Expression };
```

10.2 Safety and Determinism

- No dynamic property access, function calls, loops, regular-expression execution, network requests, dates from the wall clock, or random values.
- Maximum expression depth, operand count, and serialized size.
- Exact numeric semantics and finite-number enforcement.
- Locale-independent string comparison unless an explicit normalized comparison mode is declared.
- No secret answer values in generic issue messages or public reports.
- Canonical short-circuit behavior shared by validator, simulator, and runtime adapter.

10.3 Creator Experience

Studio should provide a visual expression builder with readable sentences, variable pickers, operator filtering, type-aware value editors, and an advanced tree view. Creators may inspect the canonical expression but do not need to hand-author JSON unless using an explicitly advanced import workflow.

11. Graph and Control-Flow Model

11.1 Canonical Edges

BlockConnection or its accepted successor is the canonical edge representation. Choice and condition configuration may contain presentation labels and option identities, but target ownership must not be duplicated in multiple structures capable of drifting apart. Drydock validates and migrates historical duplicated target fields into one authoritative graph.

11.2 Graph Analysis

- Valid Chronicle entry and chapter entry points.
- Reachable and unreachable blocks.
- Valid targets and cross-chapter transition policy.
- Every required path reaches a valid terminal state or governed loop exit.
- Cycles classified as intentional interactive loops, bounded retry loops, passive loops, or invalid automatic loops.
- Automatic-transition strongly connected components that can spin without user or provider input.
- Dead ends, orphan chapters, duplicate edges, and impossible branches.
- Optional content that is intentionally unreachable until a reveal condition.
- Terminal block multiplicity, outcome identity, and branch-to-ending map.
- Side-effect repetition risk, including duplicate artifact grants, duplicate completion, and repeatable one-shot scenes.

11.3 State-Aware Analysis

Pure graph reachability is necessary but insufficient. Drydock must perform bounded state-aware exploration for conditions and variable operations. It should identify branches that are graph-reachable but semantically impossible, conditions that always resolve the same way, and loops whose state never changes enough to exit.

When the state space exceeds configured bounds, the report must state that proof is incomplete, identify the variables or branches causing expansion, and recommend scenarios or constraints. It must not quietly label ten thousand unexplored states “probably fine.”

12. Static Analysis and Content Linting

12.1 Rule Categories

Category	Representative checks
SCHEMA	Type, enum, range, nested shape, unknown fields, migration state
STRUCTURE	Chapters, entry/terminal blocks, ordering, IDs, duplicate definitions
REFERENCE	Blocks, assets, locations, artifacts, variables, scenes, providers, outcomes
CONTROL_FLOW	Reachability, completion, cycles, automatic loops, dead ends
STATE	Declaration, initialization, types, operations, impossible conditions, side-effect duplication
CONTENT	Required copy, labels, hints, warnings, safe fallback language
ACCESSIBILITY	Alt text, captions, transcripts, keyboard path, non-motion and non-audio meaning
PRIVACY	Answers, notes, coordinates, protected assets, projection classification
PROVIDER	Availability, configuration, fallback, evidence, simulator support
PERFORMANCE	Asset size, scenario bounds, large graph, expensive media or 3D classification
COMPATIBILITY	Schema reader, platform version, package dependency, runtime feature support
TEST_COVERAGE	Branches, endings, providers, faults, accessibility modes, viewport modes

12.2 Incremental and Full Modes

Incremental validation runs on the changed block, affected edges, referenced variables, and dependent rules. Full validation freezes an immutable draft snapshot and runs all whole-Chronicle rules. The two modes share rule definitions and issue codes; incremental mode may report “pending full survey” for rules requiring global proof.

12.3 Rule Versioning

Every rule has a stable code, rule version, severity default, affected schema range, explanation, and repair guidance. Rule changes that can newly block old content require compatibility review and a migration or grandfathering policy. A renamed error message must not look like a new rule to CI.

12.4 Actionable Diagnostics

Each issue includes the Chronicle, chapter, block, field, connection, variable, scenario, or provider location; a safe human explanation; technical details for advanced users; one or more remediation paths; and whether a quick fix exists. Quick fixes must be previewable, undoable, and prohibited from rewriting story prose or destructive graph structure without explicit confirmation.

13. Media, Accessibility, Privacy, and Provider Validation

13.1 Media and Asset Rules

- Asset exists and belongs to the Chronicle or approved installed dependency.
- Required processed variant is ready and not quarantined.
- Media type and semantic role match the field contract.
- Published snapshot captures exact variants and checksums.
- Large assets receive platform-specific warnings or blockers.
- Video has poster and captions where required; audio has transcript where required; images have useful alternative text.
- 3D content declares poster/non-3D fallback and accepted performance profile through Relic Forge contracts.
- Private and Captain-only assets do not enter Player or public contexts.

13.2 Accessibility as Publishable Behavior

Accessibility is not a loose collection of warnings after the interesting work. Drydock must validate that every mandatory block and scenario has an operable keyboard path, readable status, non-color-only meaning, reduced-motion result, sound-independent meaning, and media alternative appropriate to the content. Provider-dependent

mandatory content requires an accessible fallback unless the Chronicle is explicitly and truthfully classified as restricted.

13.3 Privacy

Validation and simulation reports may include safe IDs, labels, counts, state names, and issue locations. Ordinary logs and machine reports must not include accepted answers, private prose, Captain notes, invitation secrets, raw evidence, protected storage keys, exact private locations, or unrevealed finale text. Private Creator-facing reports may display authorized content in the UI but must remain no-store, access-controlled, and excluded from broad telemetry.

13.4 Completion Providers

Every provider declares configuration schema, runtime capability, evidence schema, simulator outcomes, fault modes, retry policy, privacy class, and fallback requirements. A provider lacking a simulator may be allowed only with a required manual scenario and explicit publish limitation. `NOT_CONFIGURED` is never an alias for success, because apparently this sentence must be written into every subsystem humans build.

14. Deterministic Simulation Architecture

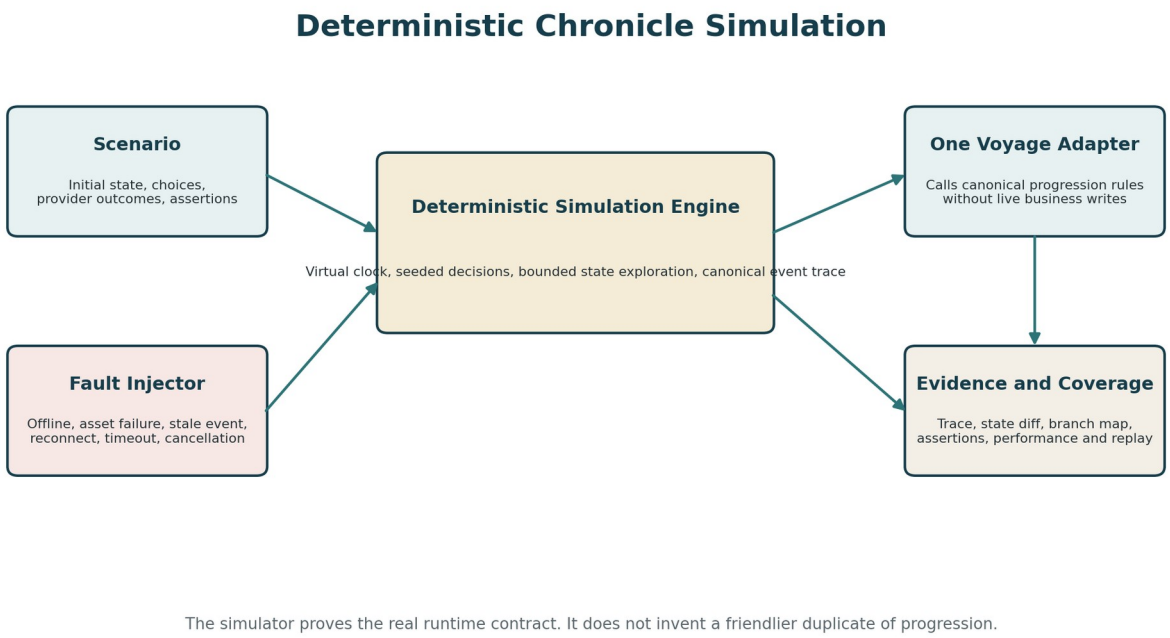

Figure 3. Scenarios and faults drive a deterministic sandbox that delegates progression decisions to the canonical runtime adapter.

14.1 Simulation Modes

Mode	Purpose
Single path	Run one Creator-selected sequence of choices and outcomes.
Scenario suite	Run a versioned set of expected paths and failure cases.
Bounded exhaustive	Explore all reachable states under declared limits and finite outcome domains.
Regression replay	Replay a previously captured deterministic trace against a newer implementation.
Differential proof	Run the same inputs through simulator and sandboxed canonical progression and compare outputs.
Performance profile	Measure step count, state size, rule duration, asset requirements, and trace limits.

14.2 Virtual Time

All timers, waits, scheduled releases, dwell periods, and timeouts use a virtual clock. Scenarios may advance time explicitly or through a governed auto-advance policy. The simulator must never sleep for ten real minutes to test a ten-minute wait block. Humanity has invented clocks; Drydock should use one responsibly.

14.3 Initial State

- Exact Chronicle draft or immutable published snapshot checksum.
- Starting chapter and block.
- Declared variable values and defaults.
- Simulated membership, crew role, Captain mode, and provider capabilities.
- Revealed content and inventory state when testing resume or historical states.
- Motion, sound, viewport, network, locale, and accessibility environment.
- Fixed random seed even when no current block uses randomness, preserving future determinism.

14.4 Trace

Each step records virtual time, current block, input, evaluated condition, variable delta, inventory delta, reveal delta, provider request/outcome, canonical event intent, presentation requirement, next block, assertion result, and safe error code. Private values are redacted according to report policy.

14.5 No Live Mutation

Simulation operates on an isolated in-memory or disposable database state owned by the run. It may not write canonical Creator drafts, published versions, real memberships, invitations, Tale Sessions, events, memories, community records, or private asset state. Preview convenience is not a license to leave synthetic artifacts wandering around the database like test raccoons.

15. Scenario Model and Coverage

15.1 Scenario Contract

```
type DrydockScenario = {
  id: string;
  revision: number;
  sourceChecksum: string;
  title: string;
  purpose: string;
  initialState: ScenarioInitialState;
  steps: ScenarioInput[];
  environment: ScenarioEnvironment;
  expected: ScenarioAssertion[];
  tags: string[];
};
```

15.2 Input Types

- Player continue or confirmation.
- Choice option selection by stable option ID.
- Text-answer result tokens such as match, no-match, or exhausted policy without recording secret answer text in reports.
- Captain approve, reject, skip, reveal, pause, resume, or governed override.
- Timer advance and timeout.
- Provider match, no-match, uncertain, unavailable, stale, duplicate, delayed, or cancelled.
- Network offline, reconnect, refresh, duplicate SSE, stale snapshot, and access revocation.
- Asset success, failure, timeout, fallback, or incompatible format.
- Presentation presented, fallback, user skip, interruption, or failure through Lanternwake contracts.

15.3 Assertions

- Current and final block or outcome.
- Variable values and allowed privacy projection.
- Inventory, artifact grant, reveal, side-quest, and completion effects.
- Event count, type, order, idempotency, and version binding.
- No duplicate award or completion.
- Expected Player-safe content present and protected content absent.
- Expected fallback, accessibility, motion, and sound behavior.
- Expected provider request and evidence disposition.
- Expected trace duration and bounded performance.

15.4 Coverage

Coverage is reported across blocks, edges, branches, outcomes, variable operations, providers, faults, endings, motion modes, sound modes, viewports, and accessibility paths. Drydock distinguishes graph coverage from state coverage and scenario coverage. Reaching both sides of a condition does not prove every relevant variable combination was tested.

15.5 Generated Scenarios

Drydock may propose minimal scenarios for uncovered edges, endings, provider outcomes, and fault classes. Generated scenarios remain drafts until accepted or saved by the Creator. The system must explain the path and assumptions; it must not fabricate secret answers or rewrite story intent.

16. Fault Injection and Environmental Simulation

Fault family	Required examples
Network	offline before input, disconnect during mutation, reconnect, slow response, duplicate event, stale snapshot
Assets	missing, 404, corrupt, unsupported, slow, fallback used, poster only, captions unavailable
Providers	unconfigured, unhealthy, timeout, uncertain, stale evidence, wrong version, revoked device
Runtime	duplicate idempotency key, concurrency conflict, cancelled request, process restart boundary
Presentation	missing target, fallback, user skip, interruption, reduced motion, sound blocked
Device	mobile viewport, landscape, low memory classification, background/foreground transition
Accessibility	keyboard only, screen-reader status, high zoom, forced colors, no audio, reduced motion
Time	midnight, timezone boundary, clock advance, scheduled start, repeated timer retry

Faults must be typed and bounded. They affect only the simulation run and may not monkey-patch global application behavior shared with another test. Every injected fault appears in the trace and report so a passing result cannot quietly depend on the fault failing to activate.

17. Runtime Fidelity and Differential Proof

17.1 Canonical Adapter

The simulator uses an adapter over canonical One Voyage progression functions. When those functions require database boundaries, Drydock uses a disposable task-owned database or an extracted pure state transition contract accepted by One Voyage. The adapter version and source commit are captured in every simulation run.

17.2 Differential Contract Tests

For every block family and important cross-block behavior, run the same initial state and input sequence through the Drydock simulator and an isolated canonical runtime. Compare state, ordered event intents, idempotency, variable changes, inventory, reveal state, outcome, and error class. Any unexplained difference is a blocker for simulator trust.

17.3 Preview Fidelity

Studio preview and play-from-here must identify whether they use real preview sessions, pure simulation, or presentation-only previews. The UI may combine them, but the distinction remains explicit. A visual preview that never executes completion rules cannot be labeled a successful scenario.

17.4 External Reality

Drydock can prove provider contracts and simulated outcomes. Real Landfall routes, Vision recognition, hardware, browser media, 3D performance, and operating-system background behavior still require their project-specific field or device tests. Drydock must surface those external evidence requirements in the publish report rather than awarding itself a medal for simulating gravity.

18. Creator Studio Experience

18.1 Drydock Workspace

Creator Studio should contain a first-class Drydock workspace reachable from the Chronicle editor, validation summary, version history, and publish flow. The workspace is one coherent product surface, not six debugging pages linked through increasingly apologetic buttons.

18.2 Primary Views

- **Overview -** readiness, blocker count, warnings, schema status, scenario status, coverage, compatibility, and last verified checksum.
- **Issues -** filterable list grouped by location and category, exact navigation, details, related rules, repairs, and waivers.
- **Graph Survey -** visual flow with unreachable, uncovered, looping, terminal, and state-dependent overlays plus nonvisual outline.
- **Variables -** declarations, types, defaults, readers, writers, operations, initialization, privacy, and rename.
- **Scenario Lab -** create, record, run, pause, step, compare, duplicate, tag, and organize deterministic scenarios.
- **Trace Inspector -** virtual timeline, inputs, evaluations, state diffs, events, provider evidence, presentation states, and assertions.
- **Coverage -** blocks, edges, outcomes, providers, faults, modes, viewports, and unproven external gates.
- **Compatibility -** schema versions, upcasts, dependencies, installed content, minimum platform, historical readers.
- **Launch Gate -** required evidence, waivers, final checksum, validation digest, and publish decision.

18.3 Issue Navigation

Selecting an issue opens the correct chapter, block, inspector field, graph edge, variable, asset, provider, scenario step, or compatibility record. Focus moves predictably and returns to the issue list. Issues remain navigable without relying on animated graph camera movement.

18.4 Repair Guidance

Repairs are classified as safe automatic, review required, manual, or architectural. Safe automatic examples include adding a missing default value defined by the current schema or removing a duplicate inert reference. Structural edge changes, variable type changes, content rewrites, provider fallback changes, and outcome remapping require explicit review. Every applied repair participates in ordinary Studio undo/redo and autosave concurrency.

18.5 Scenario Recording

Creators may record inputs while running a preview and save them as a deterministic scenario. The recorder captures stable IDs and result categories, not raw secrets or browser-specific timing. A recorded scenario can be edited into expected assertions and replayed after schema or runtime updates.

19. Validation Reports, Severity, and Repairs

19.1 Severity

Severity	Meaning	Publication effect
INFO	Explanatory evidence or optional improvement	No block
WARNING	Likely quality, compatibility, performance, or coverage concern	Visible; policy may require review
ERROR	Invalid authored state or missing required proof	Blocks publication until fixed or governed waiver permits
BLOCKER	Security, privacy, corruption, unsupported schema, runtime mismatch, or missing mandatory authority	Cannot be waived by ordinary Creator

19.2 Issue Identity

Issue identity is based on stable rule code plus semantic location, not message text. This permits message improvement without causing all previous waivers, suppressions, or CI baselines to evaporate. Reports preserve first seen, last seen, source watermark, rule version, and resolution status.

19.3 Report Forms

- Creator summary with plain-language readiness and next actions.
- Advanced technical report with rule versions, paths, state traces, and compatibility details.
- Machine-readable JSON for CLI and CI.
- Sanitized support report without private content.
- Immutable publish evidence with source and report digests.
- Diff report between validation runs or Chronicle revisions.

19.4 Waivers

A waiver records issue identity, reason, scope, authorizing role, source checksum, rule version, expiration or review condition, and whether it may carry into a later revision. Ordinary content warnings may be waived by authorized Creators. Security, privacy, corrupt schema, runtime mismatch, and missing mandatory completion proof require an Administrator or remain non-waivable. A waiver is evidence of a conscious exception, not a delete button for inconvenient reality.

20. Publishing Gates, Evidence, and Waivers

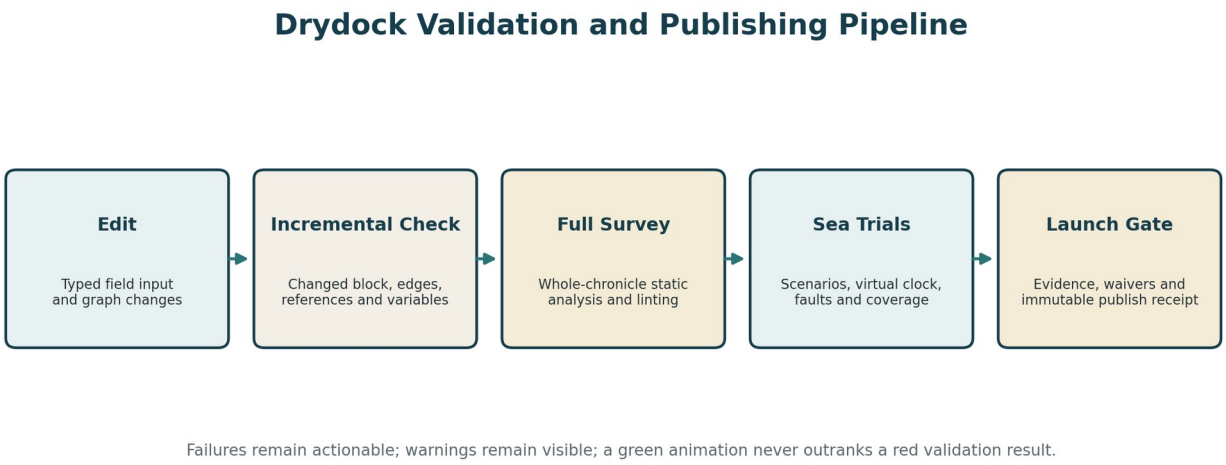

Figure 4. Drydock progresses from fast local feedback to whole-Chronicle proof and immutable launch evidence.

20.1 Gate Levels

- Edit gate: a block may be saved with issues, but impossible data should be rejected at input when safe.
- Draft readiness gate: full static validation completed for the current autosave version.
- Sea-trial gate: required scenario suite passes for the exact source checksum.
- Compatibility gate: schema readers, dependencies, providers, assets, and minimum platform are valid.
- Publication gate: all required evidence exists, unresolved errors are absent or governed, and the immutable publish transaction succeeds.
- Community gate: Harborlight package, security, licensing, accessibility, and installability rules also pass.

20.2 Publishing Evidence

```
type DrydockPublishingEvidence = {
  sourceChecksum: string;
  schemaRegistryVersion: string;
  ruleCatalogVersion: string;
  runtimeAdapterVersion: string;
  validationRunId: string;
  requiredScenarioSuiteId: string;
  scenarioRunIds: string[];
  coverageDigest: string;
  compatibilityDigest: string;
  waivers: WaiverSnapshot[];
  createdAt: string;
};
```

Evidence is immutable and tied to the exact draft snapshot published. A later edit invalidates readiness for the new checksum but does not rewrite the evidence attached to an older published version.

20.3 Truthful States

The UI must distinguish checking, needs repair, ready with warnings, trials incomplete, verified, publication pending, published, and publication failed. “Verified” means the evidence exists. “Published” means the immutable published version exists. A triumphant seal before the database commit remains forbidden, even if it is an exceptionally tasteful seal.

21. Schema Evolution and Historical Compatibility

21.1 Version Layers

- Chronicle published snapshot schema version.
- Individual Story Block schema version.
- Expression AST version.
- Variable declaration catalog version.
- Scenario schema version.
- Validation report schema version.
- Publishing evidence schema version.
- Provider contract version.

21.2 Migration Registry

Every supported migration declares source version, destination version, preconditions, transformation, warnings, data-loss status, deterministic checksum behavior, and tests. Migrations run in sequence, are idempotent where practical, and produce a preview diff before mutating a Creator draft. Published snapshots remain immutable; readers upcast them in memory or through explicitly created new drafts.

21.3 Frozen Compatibility Corpus

Maintain synthetic frozen fixtures for every supported historical snapshot and block version. Every release verifies that those fixtures still parse, upcast, validate, simulate where supported, and render a readable fallback. A current schema passing does not prove a Chronicle published six months ago remains playable.

21.4 Deprecation

A schema or rule may be deprecated only with usage inventory, migration path, minimum reader version, Creator warning, Community package impact, and removal gate. Unsupported historical content must remain identifiable and exportable where safe rather than being presented as generic corruption.

22. Cross-Project Integration Boundaries

Project	Drydock consumes	Drydock must not own
One Voyage	Block/runtime contracts, pure progression adapter, events, state transitions, publishing snapshot	Live session authority, business writes, idempotency truth
Lanternwake	Scene registry, target/fallback contracts, motion modes, presentation outcomes	Animation runtime ownership or presentation acknowledgment
Wayfarer	Canonical Creator identity, roles, preferences, privacy	Authentication, profiles, account sessions
Sealed Hold	Protected asset metadata, scan/quarantine state, authorized private preview port	Encryption, storage keys, scanner, private delivery
Harborlight	Package/install schemas, compatibility, license and community publish gates	Community listings, moderation, social data
Landfall	Waypoint/route schemas, confidence/provider simulator contracts, field-test requirements	Location engine, raw trails, authoritative arrival
Relic Forge / Artifact systems	Artifact schemas, representation and performance validators, assembly contracts	3D renderer, artifact ownership or grants
Crewline / future crew runtime	Member-scoped declarations and scenario actors when canonical	Invented per-player runtime state

22.1 Adapter Registration

Adjacent projects register validation and simulation adapters through versioned interfaces. Drydock owns orchestration, reporting, and the publishing gate; the project owning the feature owns domain semantics. This prevents Drydock from becoming a giant switch statement containing the entire platform and one developer’s rapidly fading will to live.

23. Data Model and Service Contracts

23.1 Durable Records

Concept	Purpose	Persistence guidance
DrydockValidationRun	Immutable result for a source checksum and rule catalog	Relational identity plus versioned report JSON
DrydockIssueRecord	Optional durable issue lifecycle and waiver reference	Key fields relational; message/details versioned
DrydockScenario	Creator-owned versioned scenario definition	Relational identity plus strict scenario JSON
DrydockSimulationRun	Execution status, source, environment, result, digests	Relational summary; trace stored bounded or as protected artifact
DrydockCoverageSummary	Coverage by blocks, edges, outcomes, providers, faults	Strict versioned summary
DrydockRuleWaiver	Authorized exception tied to issue/source/rule version	Relational and audited
DrydockPublishingEvidence	Immutable verification receipt for published source	Relational identity plus immutable snapshot
DrydockCompatibilityRecord	Historical reader/migration evidence	May be generated from frozen fixtures

23.2 Avoiding Database Landfill

Not every transient incremental issue requires a row. Local editor feedback may remain computed. Durable runs, scenarios, waivers, evidence, and selected traces require persistence. Versioned JSON is appropriate for bounded report structures; identity, authorization, source checksums, status, timestamps, and searchable relationships remain first-class columns.

23.3 API Families

- Validate changed block or draft snapshot.
- Run full validation and retrieve report status.
- List and inspect issues through Creator-safe projections.
- Create, update, duplicate, archive, and run scenarios.
- Pause, cancel, or inspect simulation runs.
- Retrieve trace windows, state diffs, assertions, and coverage.
- Create and revoke governed waivers.
- Check launch readiness and retrieve immutable evidence.
- Run schema migration preview and apply to a draft with optimistic concurrency.
- CLI equivalents for repository validation and CI.

23.4 Long-Running Work

Large full validation, exhaustive analysis, scenario suites, compatibility corpora, and performance runs use a durable task mechanism when they exceed request bounds. Operations are idempotent, cancellable between safe units, lease-aware, resumable after restart, and tied to source checksums. Inline execution remains acceptable for small local checks and must be labeled truthfully.

24. Security and Privacy

24.1 Threats

- Arbitrary code or expression execution through imported content.
- Path, URL, or provider reference abuse.
- Denial of service through huge graphs, expressions, scenario sets, traces, or state spaces.
- Private prose, answers, notes, coordinates, evidence, or asset keys entering logs or reports.
- IDOR against validation reports, scenarios, traces, or publishing evidence.
- Mass assignment of waiver, severity, source checksum, or authorizing role.
- Stale validation evidence reused after a draft changes.

- Simulator differences that falsely approve content the runtime rejects.
- Community packages declaring fake verification evidence.
- Cross-Chronicle reference confusion and dependency substitution.

24.2 Required Controls

- Strict Zod or equivalent schemas with bounded sizes and unknown-field policy.
- No `eval`, Function constructor, dynamic imports, arbitrary regex execution, or executable package content.
- Creator ownership and capability checks for every report, scenario, waiver, trace, and migration.
- CSRF and idempotency for mutations.
- Source checksum revalidation before run, repair, waiver, or publish.
- Redacted structured logging and safe error codes.
- Rate and concurrency limits for validation and simulation.
- Task-owned database and storage isolation for browser and integration tests.
- Private traces stored under Sealed Hold or equivalently protected storage when they contain authorized content.

24.3 Report Privacy Classes

Class	Audience	Allowed content
Creator private	Authorized Creator/collaborator	Issue locations and authorized authored labels; private values only where required in no-store UI
Support sanitized	Authorized support	IDs, versions, counts, rule codes, safe labels, correlation IDs; no prose or answers
CI machine	Repository validation	Synthetic fixtures or safe paths; no real private content
Public compatibility	Community listing/install review	Schema and capability summary only; no graph, answers, branches, or private dependencies

25. Accessibility

Drydock is an advanced Creator tool, which is not an exemption from accessibility. Complex information must be available through multiple equivalent representations.

- Graph view plus linear outline and issue list.
- Keyboard creation, editing, reordering, scenario stepping, and issue navigation.
- Visible focus and predictable restoration after dialogs or graph jumps.
- Status and run progress announced without excessive live-region noise.
- Coverage never represented only by color or spatial heat map.
- Trace timeline with tabular event alternative.
- Drag-and-drop repairs and ordering have explicit move controls.
- Reduced motion uses static state transitions and no camera flyovers while preserving location context.
- High zoom and narrow viewport support for issue panels and trace inspectors.
- Forced-colors-safe statuses and contrast.
- Accessible descriptions for diagrams, graph nodes, state changes, and failed assertions.

Drydock itself validates Chronicle accessibility, but its own interface must not become the one inaccessible tool required to fix inaccessible content. That would be almost artistic in its irony, and still unacceptable.

26. Performance and Operational Architecture

26.1 Performance Budgets

Operation	Target budget
Changed-block incremental validation	p95 under 250 ms for ordinary drafts up to 250 blocks
Full static validation	p95 under 2 seconds for 500 blocks and 1,500 edges on reference hardware
Ordinary single-path scenario	under 3 seconds and 256 MB for 1,000 simulated steps
Scenario cancellation	acknowledged within 250 ms between safe steps
Issue navigation	visible response begins within 100 ms
Persisted report load	p95 under 1 second for bounded report summaries

Operation	Target budget
Bounded exhaustive analysis	explicit state/time limits; partial evidence rather than unbounded execution

Budgets are reference goals and must be measured on recorded hardware and production builds. Correctness, privacy, and determinism are not removed to meet a timing target. Expensive analysis becomes an explicit durable run rather than an invisible browser freeze.

26.2 Incremental Dependency Index

Maintain indexes from blocks and fields to edges, variables, assets, providers, scenes, scenarios, and rules. A changed variable type should invalidate dependent conditions and scenarios, not every unrelated photograph and subtitle in the Chronicle.

26.3 Resource Limits

- Maximum blocks, edges, variables, expression depth, scenario steps, scenario count, trace size, state count, run duration, and concurrent runs.
- Graceful warnings before hard limits where possible.
- Streamed or paginated trace retrieval.
- Document-hidden and unmounted UI work cancellation.
- No full scenario suite automatically triggered on every keystroke.

26.4 Observability

Track safe metrics such as validation duration, rule duration, issue counts by code, scenario duration, state count, cancellation, coverage percentage, compatibility failures, and publish-gate outcomes. Do not use Chronicle titles, prose, answers, variable values, or private IDs as metric labels.

27. Testing and Acceptance Matrix

27.1 Required Test Families

Family	Required proof
Schema	Every block version; valid, invalid, boundary, unknown fields, canonicalization and migration
Variables and expressions	Types, scopes, initialization, operations, rename, constants, depth and unsafe input
Graph	Reachability, terminals, cycles, auto loops, optional branches, cross-chapter policy, side effects
Lint rules	Stable issue identity, location, severity, repair, version and waiver behavior
Simulation	Determinism, virtual time, all block families, assertions, cancellation and trace
Differential	Simulator and canonical One Voyage produce semantically identical state and event intents
Faults	Network, provider, asset, reconnect, duplicate, stale, presentation, accessibility and time
Privacy and security	IDOR, CSRF, mass assignment, code injection, report redaction, source checksum
Compatibility	Frozen historical snapshots, upcasters, packages, minimum reader and deprecated schemas
Studio	Issue navigation, graph alternatives, variables, scenarios, traces, coverage, waivers, publish gate
Accessibility	Axe, keyboard, focus, zoom, forced colors, reduced motion and screen-reader status
Performance	Budgets, state limits, cancellation, memory, large synthetic Chronicle and run concurrency
Repository gate	Format, lint, types, language, unit, E2E, build, migrations, private scans and CI

27.2 Synthetic Chronicle Corpus

Commit a library of clearly synthetic Chronicles representing linear success, every block family, branching endings, intentional loops, impossible loops, missing references, variable errors, privacy leaks, accessibility gaps, provider faults, asset faults, compatibility migrations, performance limits, and expected warnings. Never use real private Chronicle content for broad repository tests.

27.3 Property and Mutation Testing

Use property-based generation within strict bounds for graph, expressions, and state operations. Apply controlled mutations to valid synthetic Chronicles, such as deleting an edge, changing a variable type, duplicating a grant,

removing a caption, or altering a target, and prove the intended rule detects the change. This tests whether the alarm works, not only whether it remains quiet in an empty building.

27.4 Browser Matrix

- Desktop and mobile Creator Studio layouts.
- Keyboard-only validation and scenario workflows.
- Issue-to-field and issue-to-graph navigation.
- Scenario run, cancel, replay, compare, and trace inspection.
- Reduced motion, no audio, high zoom, forced colors, and Axe.
- Concurrent draft edit invalidating a report or run.
- Publication success only after authoritative evidence and immutable publish commit.

28. Implementation Phases

Project Drydock should be divided into four phases. Three phases would compress static analysis, simulation, Studio integration, publishing, compatibility, and release hardening into one oversized middle phase. Four provides coherent architectural and acceptance boundaries without stretching the program into another six-part saga that develops its own weather system.

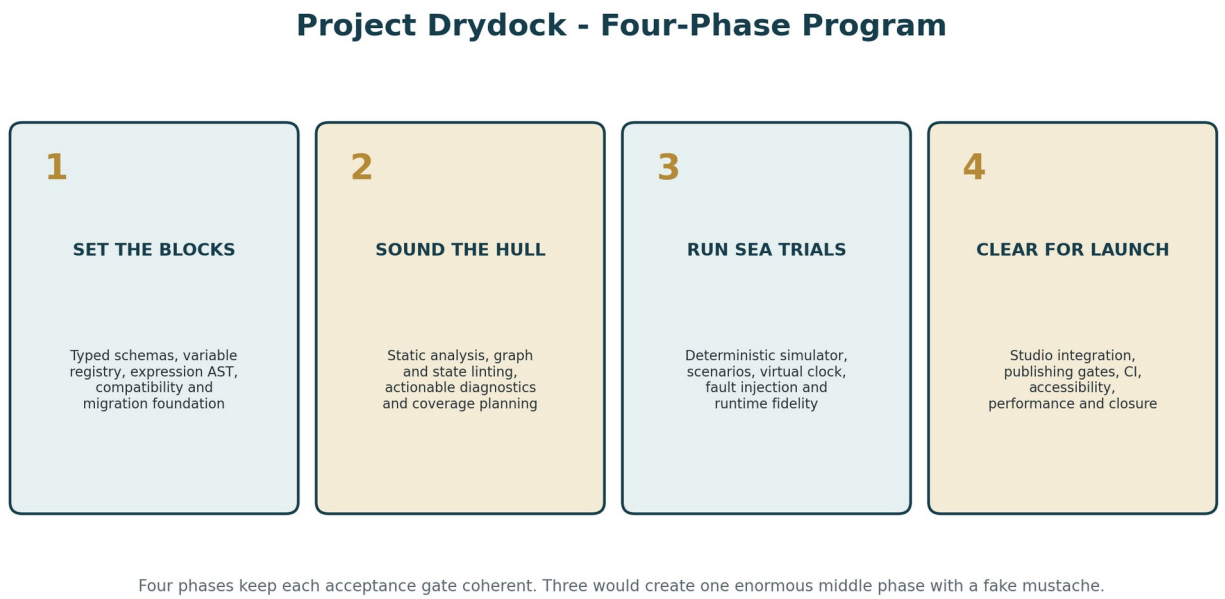

Figure 5. Four phases separate typed foundations, static proof, executable proof, and release integration.

28.1 Phase 1: Set the Blocks

Formal scope: Typed Authoring Contracts, Variable Registry, Expression System, and Schema Evolution Foundation.

- Current-state repository and authoring inventory.
- Versioned Drydock block contract and strict schemas for all current Story Block types.
- Separate configuration, presentation, completion, connection, asset, provider, and accessibility contracts.
- Canonical parsing and serialization.
- Typed variable registry, scopes, operations, usage index, and rename propagation.
- Typed expression AST and visual-builder contract.
- Schema migration registry and frozen historical fixture foundation.
- Incremental validation infrastructure and stable issue model.
- Compatibility adapters for current generic configuration and duplicated target fields.

- CLI schema validation and baseline tests.

> **Phase 1 Completion Gate**
>
> Every current block type has a strict, versioned contract; current drafts can be parsed or receive exact migration diagnostics; variables and expressions are typed; historical fixtures remain readable; no live progression behavior changes.

28.2 Phase 2: Sound the Hull

Formal scope: Whole-Chronicle Static Analysis, Graph and State Linting, Diagnostics, and Repair Guidance.

- Canonical graph builder and edge migration.
- Reachability, terminal, cycle, automatic-loop, state-aware branch, and side-effect analysis.
- Variable initialization, type-flow, impossible condition, unused state, and privacy analysis.
- Complete rule catalog across schema, references, assets, accessibility, privacy, provider, performance, and compatibility.
- Incremental dependency invalidation.
- Creator issue navigator, graph overlays, variable explorer, rule details, and undoable safe repairs.
- Validation report persistence, diffing, sanitized support view, and machine JSON.
- Waiver foundation with non-waivable classes.
- Synthetic invalid Chronicle corpus and rule mutation tests.

> **Phase 2 Completion Gate**
>
> A full Chronicle survey identifies every governed static defect with stable issue identity and actionable location; state and control-flow proof is bounded and truthful; Creator repairs are safe and reversible; publication remains blocked on unresolved required errors.

28.3 Phase 3: Run Sea Trials

Formal scope: Deterministic Chronicle Simulation, Scenario Laboratory, Fault Injection, Coverage, and Runtime Fidelity.

- Canonical One Voyage simulation adapter and differential contract suite.
- Virtual clock, deterministic seed, isolated state, trace and assertions.
- Single-path, suite, bounded exhaustive, regression replay, and differential modes.
- Creator-authored and generated scenarios.
- Fault injection for network, provider, asset, runtime, presentation, device, accessibility, and time.
- Block, edge, ending, state, provider, fault, mode, and viewport coverage.
- Scenario Lab, trace inspector, state diff, expected outcome editor, compare and replay.
- Durable cancellable runs for large suites.
- Performance and state-explosion limits.
- Regression scenario corpus covering every canonical block family.

> **Phase 3 Completion Gate**
>
> For the same source, inputs, and environment, simulation is deterministic and matches canonical One Voyage behavior; required scenarios and faults produce reproducible evidence; coverage gaps are explicit; no simulation writes live business state.

28.4 Phase 4: Clear for Launch

Formal scope: Complete Studio Integration, Publishing Gates, Compatibility Proof, CI, Accessibility, Performance, and Program Closure.

- Integrated Drydock workspace and launch-readiness experience.
- Required scenario-suite policy by Chronicle capabilities and installed providers.
- Immutable publishing evidence and exact-checksum gate.
- Governed waiver UI and audit.
- Historical compatibility corpus, upcasters, deprecation gates, and migration preview.
- Harborlight install/remix validation integration.

- Landfall, provider, artifact, and Lanternwake external-evidence summaries.
- Hosted CI commands and branch protection integration where infrastructure permits.
- Complete browser, accessibility, performance, privacy, security, restart, and production-build validation.
- Operations and support documentation, metrics, final acceptance ledger, and project completion receipt.

> **Phase 4 Completion Gate**
>
> Creator Studio, CLI, CI, and publishing agree on one readiness decision; immutable evidence is tied to the exact published source; historical versions remain supported; accessibility and performance budgets pass; all locally attainable requirements are validated and documented.

28.5 Relative Effort and Parallelization

Phase	Approximate share	Parallel work after shared contracts freeze
1. Set the Blocks	20-25%	Block schema lanes, migration fixtures, variable/expression UI after registry contracts
2. Sound the Hull	25-30%	Graph analysis, rule families, issue UI, synthetic corpus with serialized shared rule catalog
3. Run Sea Trials	30-35%	Scenario UI, fault adapters, trace/coverage UI after simulation core and One Voyage adapter freeze
4. Clear for Launch	20-25%	Compatibility corpus, CI, browser/accessibility, docs after publishing evidence contract freeze

29. Final Acceptance Criteria

Gate	Required result
One contract	Studio, validation, simulation, publishing, imports, and historical readers use the same versioned authoring contracts.
Strict schemas	Every current block type enforces types, enums, ranges, nested shapes, references, and cross-field rules.
Typed state	Variables and expressions are declared, typed, bounded, deterministic, and free of arbitrary code.
Graph truth	Reachability, endings, loops, state-dependent branches, and side-effect risks are analyzed truthfully.
Actionable lint	Every governed issue has a stable code, exact location, explanation, remediation and severity.
Deterministic trials	Same source and scenario produce the same trace, state and result.
Runtime fidelity	Differential tests show the simulator agrees with canonical One Voyage semantics.
No live mutation	Validation and simulation never change real progression, memberships, events, invitations, private assets or Community records.
Coverage	Required branches, endings, providers, faults, motion, sound, viewport and accessibility modes have explicit evidence or explicit unproven status.
Publish evidence	Every published version has immutable Drydock evidence tied to its exact source checksum and schema versions.
Historical compatibility	Every supported historical fixture parses, upcasts or renders a governed fallback and remains version-pinned.
Privacy	Reports, traces, logs and public projections do not expose answers, private prose, Captain notes, evidence, storage keys or private locations.
Security	No arbitrary code, unsafe expression, IDOR, mass assignment, stale evidence reuse or unbounded input passes.
Accessibility	Drydock UI and mandatory Chronicle paths pass keyboard, focus, zoom, contrast, reduced-motion, sound-independent and automated checks.
Performance	Incremental, full, scenario and report operations meet recorded budgets or enter truthful durable-run modes.
Integration	Lanternwake, Sealed Hold, Harborlight, Landfall and artifact providers integrate through declared contracts without duplicated ownership.
Validation	Focused tests, differential tests, browser acceptance, accessibility, migrations, privacy scans, production build and repository gate pass.
Documentation	Governing, implementation, testing, compatibility, operations and completion records match actual behavior.

30. Recommended Technical Baseline

Area	Recommended baseline
Language and schemas	Strict TypeScript and Zod 4 or current accepted equivalent
Graph analysis	Purpose-built adjacency/SCC/data-flow algorithms; no heavy external engine required initially
Expressions	Versioned typed AST with shared parser/evaluator and no arbitrary scripting

Area	Recommended baseline
Simulation	Canonical One Voyage adapter, pure state where practical, disposable SQLite for integration fidelity
Virtual time	Explicit Drydock clock interface; no real sleeps for authored timers
Property tests	Fast-check or equivalent may be added if justified; deterministic custom generators are acceptable
Persistence	Prisma SQLite/MySQL parity for durable scenarios, runs, waivers and evidence
Long work	Existing accepted durable task/outbox primitive or Drydock-owned bounded runner
UI	React 19, current Creator Studio patterns, Motion for layout/presence, Lanternwake for substantial scenes
Reports	Versioned JSON plus explicit Creator/support/public projections
CLI	Repository-standard TypeScript scripts with sanitized JSON output and nonzero failure codes
CI	Current repository validation plus focused Drydock validation/simulation/compatibility commands

30.1 Suggested Source Organization

```
src/drydock/
  contracts/
  schema/
  variables/
  expressions/
  graph/
  rules/
  simulation/
  scenarios/
  faults/
  coverage/
  evidence/
  publishing/
  compatibility/
  projections/

src/components/studio/drydock/
scripts/drydock/
tests/drydock/
```

The exact structure should follow current repository conventions. The governing requirement is cohesive ownership and one shared contract layer, not a ritual recreation of this directory tree.

31. Governance and Change Control

31.1 Governing Records

- Project Drydock governing document.
- Per-phase design records, test plans, validation records, manifests and completion receipts.
- Block schema registry and migration catalog.
- Rule catalog and severity policy.
- Scenario schema and required suite policy.
- Runtime fidelity matrix.
- Historical compatibility ledger.
- Publishing evidence schema.

31.2 Rule Changes

A rule change must identify affected block and snapshot versions, whether it changes severity, whether old evidence remains valid, required fixture updates, migration or waiver policy, and Creator-facing release notes. Adding a blocker because one current test happened to fail is not governance; it is debugging wearing a tie.

31.3 New Block or Provider Checklist

1. Define versioned authoring schemas and canonical defaults.

2. Define graph, state, asset, privacy, accessibility, performance and compatibility rules.
3. Define simulation adapter, input outcomes, faults and assertions.
4. Define runtime adapter ownership and differential tests.
5. Define Creator inspector and Drydock issue locations.
6. Define historical migration and minimum reader behavior.
7. Define required scenario-suite additions and publishing evidence.
8. Define public or Community projection restrictions.

31.4 Project Closure

Project Drydock is complete only after Phase 4 produces a formal completion receipt showing zero unmapped mandatory contract families, zero unexplained simulator/runtime differences, zero unsupported current block types, and zero unresolved blocker issues in the synthetic acceptance corpus. Future block types extend the governed platform; they do not reopen the entire project unless they expose a foundational defect.

32. Glossary

Term	Meaning
Authoring contract	Versioned schemas and metadata governing how a Chronicle element is created, stored, validated, simulated and read.
Canonicalization	Conversion of valid equivalent input into one deterministic representation.
Static analysis	Reasoning about a Chronicle without executing a specific path.
State-aware analysis	Bounded reasoning about graph paths and variable values together.
Scenario	Versioned deterministic set of initial state, inputs, environment and expected assertions.
Sea trial	Execution of a scenario or suite in the Drydock simulator.
Virtual clock	Simulation-controlled time source used for timers, waits and scheduling.
Fault injection	Deliberate simulated environmental or runtime failure.
Trace	Ordered safe record of simulation inputs, evaluations, state changes, event intents and assertions.
Coverage	Evidence of which blocks, edges, states, outcomes, providers, faults and modes were exercised.
Differential proof	Comparison of Drydock and canonical One Voyage results for identical inputs.
Publishing evidence	Immutable verification receipt tied to an exact source checksum.
Waiver	Governed, audited acceptance of a specific issue under a specific source and rule version.
Upcast	Read-time conversion from an older supported schema to the current in-memory representation.
External evidence	Real-device or provider proof owned by another project and summarized by Drydock.

33. References

1. Kgray44/treasurehuntSoT, repository baseline 676b21ed030a5470d4ea0a36c0688ed3ecb161e5.
2. `src/chronicle/block-registry.ts`, current Story Block registry and common validation-schema builder.
3. `src/chronicle/validation.ts`, current draft validation, asset checks, graph reachability, and completion-path analysis.
4. `docs/chronicle-studio.md`, current authoring, publishing, preview, assets, and progression overview.
5. Project One Voyage governing and completion records, canonical authoring/publishing/runtime authority.
6. Project Lanternwake governing and completion records, presentation truth and runtime ownership.
7. Project Sealed Hold governing records, protected private content and assets.
8. Project Harborlight Community Harbor Governing Document, package/install/publication integration boundaries.
9. Project Landfall Governing Document, provider boundaries, field evidence, and four-phase governing pattern.
10. Project Wayfarer governing records, canonical identity, preferences, privacy and personal history ownership.

> **Final Governing Rule**
>
> A Chronicle is ready to leave Drydock only when the same content, the same schema versions, the same runtime rules, and the same declared scenarios produce reproducible evidence that it can launch safely. “It worked when I tried it” remains welcome anecdotal evidence and an unacceptable release gate.